Lab 2 Report:

Iris Classification with Regression

Nate Chaoweeraprasit

In [46]:
```
%pip install sklearn
```

```
Defaulting to user installation because normal site-packages is not wri
teable
Requirement already satisfied: sklearn in /Users/natchatemp/Library/Pyt
hon/3.9/lib/python/site-packages (0.0)
Requirement already satisfied: scikit-learn in /Users/natchatemp/Librar
y/Python/3.9/lib/python/site-packages (from sklearn) (1.6.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /Users/natchatem
p/Library/Python/3.9/lib/python/site-packages (from scikit-learn->sklea
rn) (3.6.0)
Requirement already satisfied: numpy>=1.19.5 in /Users/natchatemp/Libra
ry/Python/3.9/lib/python/site-packages (from scikit-learn->sklearn) (2.
0.2)
Requirement already satisfied: joblib>=1.2.0 in /Users/natchatemp/Libra
ry/Python/3.9/lib/python/site-packages (from scikit-learn->sklearn) (1.
5.3)
Requirement already satisfied: scipy>=1.6.0 in /Users/natchatemp/Librar
y/Python/3.9/lib/python/site-packages (from scikit-learn->sklearn) (1.1
3.1)
WARNING: You are using pip version 21.2.4; however, version 26.0.1 is a
vailable.
You should consider upgrading via the '/Library/Developer/CommandLineTo
ols/usr/bin/python3 -m pip install --upgrade pip' command.
Note: you may need to restart the kernel to use updated packages.
```

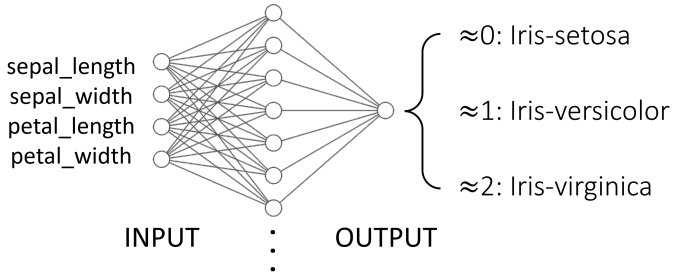
In [47]:
```
# Import neccessary packages

%matplotlib inline

import numpy as np
import matplotlib.pyplot as plt

import torch
```

In [48]:
```
from IPython.display import Image # For displaying images in colab jup
```

In [49]:
```
Image('lab2_exercise1.PNG', width = 1000)
```

Out[49]:

Exercise 1: Iris Classification using Regression



In this exercise, you will train a neural network with a single hidden layer consisting of linear neurons to perform regression on iris datasets.

Your goal is to achieve a training accuracy of >90% under 50 epochs.

You are free to experiment with different data normalization methods, size of the hidden layer, learning rate and epochs.

You can round the output value to an integer (e.g. 0.34 -> 0, 1.78 -> 2) to compute the model accuracy.

Demonstrate the performance of your model via plotting the training loss and printing out the training accuracy. 42

Prepare Data

In [50]:

```
# scaling function
def scale_data(arr):

    # exception gate; ensure array is 2D array
    if arr.ndim != 2:
        raise ValueError("array isn't 2D")

    # create copy so we don't affect original array
    scaled_data = arr.copy()

    # iterate by column
    for i in range(scaled_data.shape[1]):

        # for each column, find mean and stdev
        column_mean = np.mean(scaled_data[:, i])
        column_stdev = np.std(scaled_data[:, i])

        # then do (x-mean)/std to find scaled value at each point
        scaled_data[:, i] = (scaled_data[:, i] - column_mean) / column

    return scaled_data
```

In [51]:

```
from sklearn.datasets import load_iris

# iris dataset is available from scikit-learn package
iris = load_iris()

# Load the X (features) and y (targets) for training
x_train = iris['data']
```

```
y_train = iris['target']

# Load the name labels for features and targets
feature_names = iris['feature_names']
names = iris['target_names']

# Feel free to perform additional data processing here (e.g. standard
x_train = scale_data(x_train)
```

In [52]:
```
# Print the first 10 training samples for both features and targets

print(x_train[:10, :], y_train[:10])
```

```
[[-0.90068117  1.01900435 -1.34022653 -1.3154443 ]
 [-1.14301691 -0.13197948 -1.34022653 -1.3154443 ]
 [-1.38535265  0.32841405 -1.39706395 -1.3154443 ]
 [-1.50652052  0.09821729 -1.2833891  -1.3154443 ]
 [-1.02184904  1.24920112 -1.34022653 -1.3154443 ]
 [-0.53717756  1.93979142 -1.16971425 -1.05217993]
 [-1.50652052  0.78880759 -1.34022653 -1.18381211]
 [-1.02184904  0.78880759 -1.2833891  -1.3154443 ]
 [-1.74885626 -0.36217625 -1.34022653 -1.3154443 ]
 [-1.14301691  0.09821729 -1.2833891  -1.44707648]] [0 0 0 0 0 0 0 0 0
0]
```

In [53]:
```
# Print the dimensions of features and targets

print(x_train.shape, y_train.shape)
```

```
(150, 4) (150,)
```

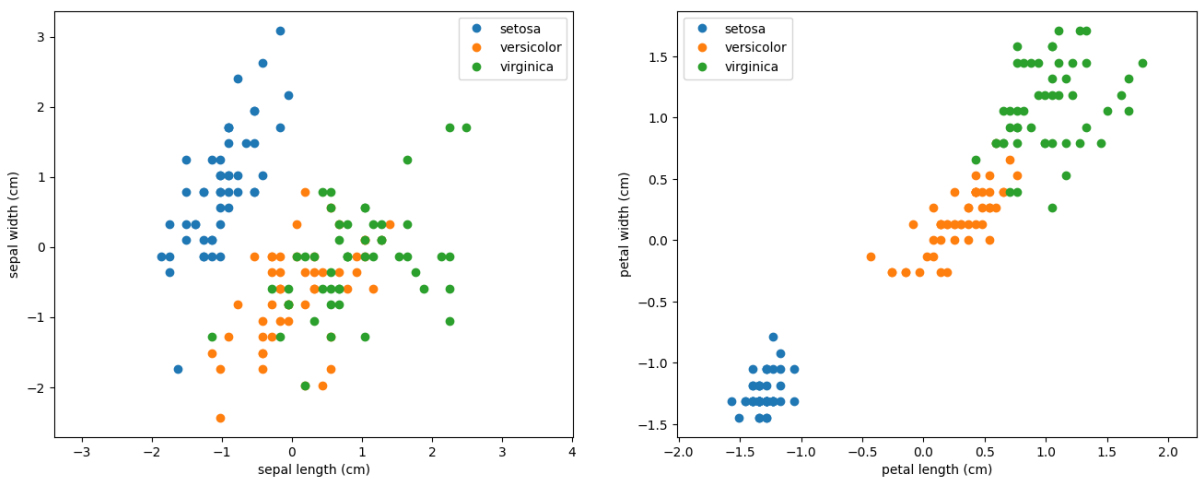
In [54]:
```
# feature_names contains name for each column in X_train
# For targets, 0 -> setosa, 1 -> versicolor, 2 -> virginica

print(feature_names, names)
```

```
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal w
idth (cm)'] ['setosa' 'versicolor' 'virginica']
```

In [ ]:
```
# We can visualize the dataset before training

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# enumerate picks up both the index (0, 1, 2) and the element ('setosa
# loop 1: target = 0, target_name = 'setosa'
# loop 2: target = 1, target_name = 'versicolor' etc

for target, target_name in enumerate(names):

    # Subset the rows of X_train that fall into each flower category u
    x_plot = x_train[y_train == target]

    # Plot the sepal length versus sepal width for the flower category
```

```
    ax1.plot(x_plot[:, 0], x_plot[:, 1], linestyle='none', marker='o',

# Label the plot
ax1.set_xlabel(feature_names[0])
ax1.set_ylabel(feature_names[1])
ax1.axis('equal')
ax1.legend()

# repeat for petal length versus petal width
for target, target_name in enumerate(names):

    x_plot = x_train[y_train == target]

    ax2.plot(x_plot[:, 2], x_plot[:, 3], linestyle='none', marker='o',

ax2.set_xlabel(feature_names[2])
ax2.set_ylabel(feature_names[3])
ax2.axis('equal')
ax2.legend()
```

Out[ ]: <matplotlib.legend.Legend at 0x13cf47f40>



Define Model

In [ ]:
```
class irisClassification(torch.nn.Module):

    def __init__(self, input_dim, hidden_dim, output_dim):

        super(irisClassification, self).__init__()

        self.layer1 = torch.nn.Linear(input_dim, hidden_dim)
        self.layer2 = torch.nn.Linear(hidden_dim, output_dim)

    def forward(self, x):

        # two linear layers
        layer1_out = self.layer1(x)
```

```
        layer2_out = self.layer2(layer1_out)

        return layer2_out
```

Define Hyperparameters

In [57]:
```
model = irisClassification(input_dim = 4, hidden_dim = 20, output_dim

learning_rate = 0.01
epochs  = 50

# We will use gradient descent for our optimizer and Mean Squared Erro
loss_func = torch.nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr = learning_rate)

if torch.cuda.is_available():
    model.cuda()
```

Identify Tracked Values

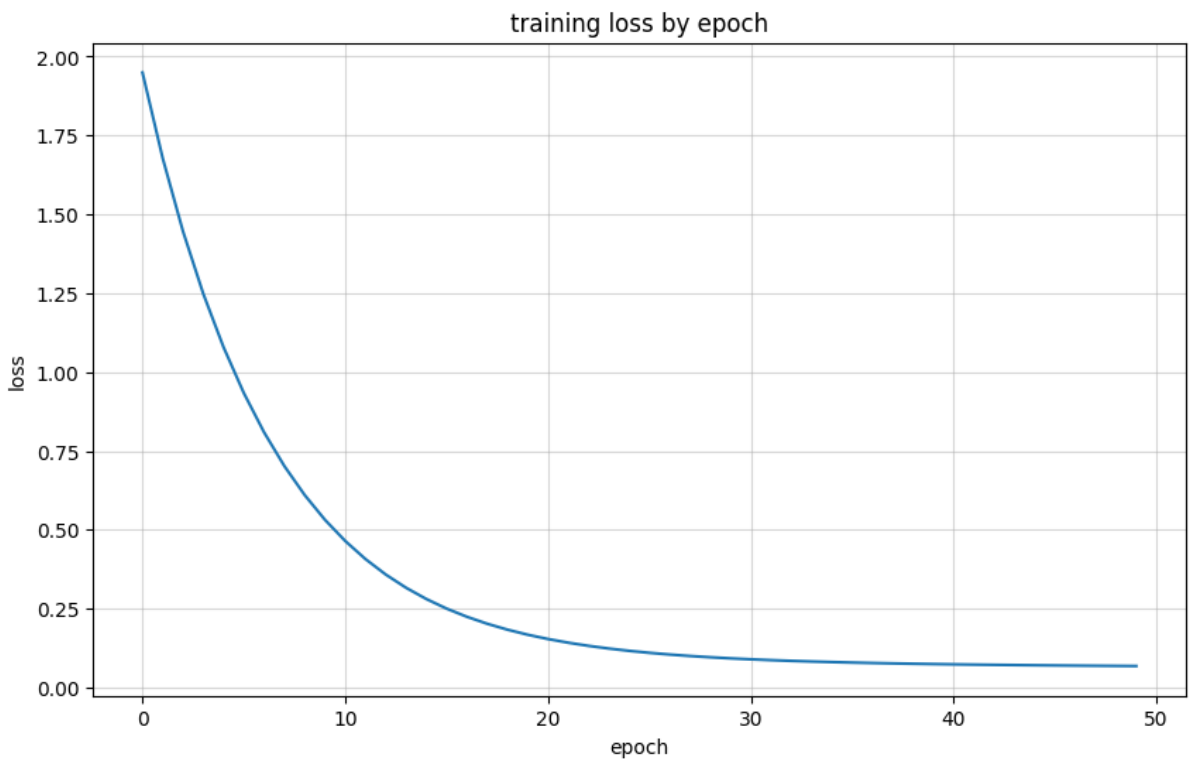
Train Model

In [ ]:
```
train_loss_list = []
inputs = torch.from_numpy(x_train).float()
targets = torch.from_numpy(y_train).float()

for epoch in range(epochs):

    optimizer.zero_grad()
    outputs = model(inputs)

    # unsqueeze to make targets have same shape as outputs (50, 1)
    loss = loss_func(outputs, targets.unsqueeze(1))

    # save loss value to train_loss_list we defined
    train_loss_list.append(loss.item())

    # compute how much changes to be made to weights/biases
    loss.backward()

    # update weights/biases according to learning rate
    optimizer.step()

    print(f"epoch {epoch:02d} | loss: {loss.item():.6f}")
```

```
epoch 00 | loss: 1.949588
epoch 01 | loss: 1.677485
epoch 02 | loss: 1.445887
epoch 03 | loss: 1.248020
epoch 04 | loss: 1.078576
epoch 05 | loss: 0.933287
epoch 06 | loss: 0.808655
epoch 07 | loss: 0.701760
epoch 08 | loss: 0.610128
epoch 09 | loss: 0.531646
epoch 10 | loss: 0.464492
epoch 11 | loss: 0.407089
epoch 12 | loss: 0.358068
epoch 13 | loss: 0.316242
epoch 14 | loss: 0.280580
epoch 15 | loss: 0.250190
epoch 16 | loss: 0.224301
epoch 17 | loss: 0.202248
epoch 18 | loss: 0.183460
epoch 19 | loss: 0.167448
epoch 20 | loss: 0.153792
epoch 21 | loss: 0.142136
epoch 22 | loss: 0.132176
epoch 23 | loss: 0.123654
epoch 24 | loss: 0.116351
epoch 25 | loss: 0.110082
epoch 26 | loss: 0.104689
epoch 27 | loss: 0.100040
epoch 28 | loss: 0.096023
epoch 29 | loss: 0.092543
epoch 30 | loss: 0.089520
epoch 31 | loss: 0.086886
epoch 32 | loss: 0.084583
epoch 33 | loss: 0.082564
epoch 34 | loss: 0.080787
epoch 35 | loss: 0.079217
epoch 36 | loss: 0.077825
epoch 37 | loss: 0.076586
epoch 38 | loss: 0.075478
epoch 39 | loss: 0.074483
epoch 40 | loss: 0.073585
epoch 41 | loss: 0.072772
epoch 42 | loss: 0.072032
epoch 43 | loss: 0.071355
epoch 44 | loss: 0.070733
epoch 45 | loss: 0.070159
epoch 46 | loss: 0.069627
epoch 47 | loss: 0.069132
epoch 48 | loss: 0.068669
epoch 49 | loss: 0.068234
```

Visualize and Evaluate Model

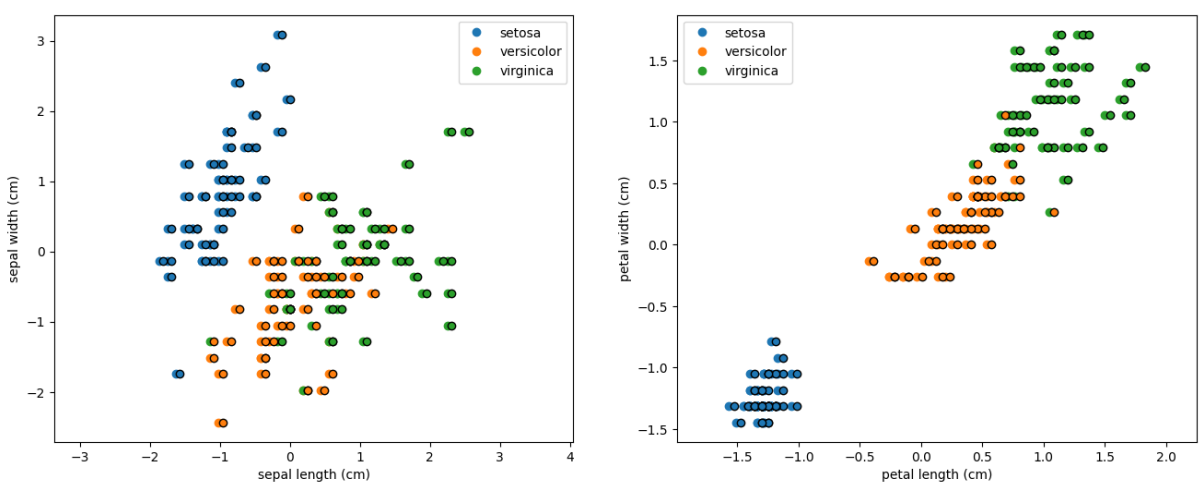
In [59]:
```
# Plot your training loss throughout the training
# Include proper x and y labels for the plot

plt.figure(figsize = (10,6))
plt.plot(train_loss_list)
plt.xlabel('epoch')
plt.ylabel('loss')
plt.title('training loss by epoch')
plt.grid(alpha = 0.5)
plt.show()
```



In [ ]:
```
# We can visualize the dataset before training

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# enumerate picks up both the index (0, 1, 2) and the element ('setosa
# loop 1: target = 0, target_name = 'setosa'
# loop 2: target = 1, target_name = 'versicolor' etc

predicted = predicted.squeeze() # remove extra dimension from predicte

color = ['tab:blue', 'tab:orange', 'tab:green']

for (target, target_name), c in zip(enumerate(names), color):

    # Subset the rows of X_train that fall into each flower category u
    x_plot_actual = x_train[y_train == target]
```

```
    # Plot the sepal length versus sepal width for the flower category
    ax1.plot(x_plot_actual[:, 0], x_plot_actual[:, 1], linestyle='none
             marker='o', label=target_name, color = c)

for (target, target_name), c in zip(enumerate(names), color):

    x_plot_predicted = x_train[predicted == target]
    # put borders around points
    ax1.plot(x_plot_predicted[:, 0]+0.06, x_plot_predicted[:, 1], line
             marker='o', markeredgecolor = 'black',color = c)

# Label the plot
ax1.set_xlabel(feature_names[0])
ax1.set_ylabel(feature_names[1])
ax1.axis('equal')
ax1.legend()

# Repeat the above process but with petal length versus petal width
for (target, target_name), c in zip(enumerate(names), color):

    # Subset the rows of X_train that fall into each flower category u
    x_plot_actual = x_train[y_train == target]
    # Plot the sepal length versus sepal width for the flower category
    ax2.plot(x_plot_actual[:, 2], x_plot_actual[:, 3], linestyle='none
             marker='o', label=target_name, color = c)

for (target, target_name), c in zip(enumerate(names), color):

    x_plot_predicted = x_train[predicted == target]
    ax2.plot(x_plot_predicted[:, 2]+0.04, x_plot_predicted[:, 3], line
             marker='o', markeredgecolor = 'black', color = c)

ax2.set_xlabel(feature_names[2])
ax2.set_ylabel(feature_names[3])
ax2.axis('equal')
ax2.legend()
```

Out[ ]: <matplotlib.legend.Legend at 0x13d1871c0>



Black outlined points are the predicted classification for each point, the markers underneath it are the actual classification.

The results are pretty close!

In [65]:
```
# test reliability of model by comparing the model predictions with ta
with torch.no_grad():

    # compare model predictions with actual outputs to compute trainin
    outputs = model(inputs)
    predicted = torch.round(outputs)

    # tally all correct predictions and divide by total # samples
    correct = 0
    for i in range(predicted.shape[0]):
        if predicted[i] == targets[i]:
            correct += 1

    total = targets.size(0)
    accuracy = correct / total
    print(f'training accuracy: {accuracy*100:.4f}')
```

```
training accuracy: 93.3333
```